

통계적 기계학습

차량 데이터 디젤과 가솔린 분류 모델 성능 평가



학번	12243726	이름	송민서
학번	12243734	이름	이민정

목차

1. 서론

가. 연구 배경

2. 본론

나. 데이터셋 소개

1. 데이터 설명
2. 변수 소개
3. 변수 선정 이유

다. 데이터 정제

1. 결측치 제거
2. 이상치 제거
 - 가) Scaling(표준화)
 - 나) Mahalanobis Distance 기반 이상치 검출
 - 다) 이상치 처리 전략
 - 라) 최종 판단

라. 회귀 가정 진단

1. 다중공선성
 - 가) VIF 분석
2. 최종 변수 선정
 - 가) 설명변수 그룹화
 - 나) 반응변수 재정의(Labeling)

마. 데이터 분할 및 검증

1. K-Fold 교차검증 기반 학습 · 평가 데이터 구성

바. 여러 모델 비교

1. OLS (Ordinary Least Squares)
2. WLS (Weighted Least Squares)
3. OLS와 WLS 성능 비교
4. Elastic Net

5. 선형 모델의 한계

6. 불균형 데이터에서의 딜레마

사. 모델별 성능 분석 및 Bias-Variance Trade-off

1. LDA (Linear Discriminant Analysis)

2. QDA (Quadratic Discriminant Analysis)

3. SVM (Support Vector Machine)

아. 성능 평가 및 비교

3. 결론

1. 서론

가. 연구배경

최근 자동차 산업에서는 환경 규제 강화와 연료 효율성에 대한 관심 증가로 인해 차량의 연료 소비 특성과 CO2 배출량 분석의 중요성이 커지고 있다. 특히 엔진 크기 (Engine Size), 실린더 수(Cylinders), 연비(Fuel Consumption), CO2 배출량 등의 차량 성능 변수들은 연료 종류(Fuel Type)에 따라 서로 다른 특성을 나타낼 가능성이 있다. 이러한 변수들의 패턴을 활용하여 차량의 연료 유형을 분류할 수 있다면 차량 특성 분석 및 환경 관련 연구에 활용될 수 있다.

본 연구에서는 자동차 데이터를 이용하여 차량의 성능 및 연비 관련 변수들을 기반으로 연료 유형을 분류하고자 하였다. 설명변수로는 Engine Size, Cylinders, Fuel Consumption City, Fuel Consumption Hwy, CO2 Emissions, Transmission Group 등을 사용하였으며, 반응변수는 연료 유형(Fuel Type)으로 설정하였다. 또한 연료 유형 간 특성 차이를 분석하고, 이를 효과적으로 분류할 수 있는 분류모형을 비교하기 위해 LDA, QDA, SVM, Elastic Net Logistic Regression 모형을 적용하였다.

2. 본론

가. 데이터셋 소개

1) 데이터 설명

본 연구에서 사용된 데이터셋은 차량의 물리적 특성과 연료 정보를 기반으로 이산화탄소 배출량을 측정한 차량 탄소 배출 데이터(co2.csv)이다.

해당 데이터셋은 다양한 제조사의 차량을 대상으로 하며, 엔진 특성, 연료 유형, 차량 구조 등 여러 요인을 포함하고 있다.

특히 본 데이터는 실제 차량의 성능 및 연료 소비와 밀접하게 연관된 변수들로 구성되어 있으며, 동일한 조건에서 다양한 차량 유형이 포함되어 있어, 차량의 구조적 차이 및 연료 특성에 따른 배출량 변화를 비교·분석할 수 있는 장점을 가진다.

2) 변수 소개

본 연구에서는 차량의 연료 종류(Fuel Type)를 분류하기 위해 차량의 구조적 특성, 엔진 특성, 주행 효율 특성 및 배출 특성을 반영할 수 있는 변수들을 선정하였다.

변수명	변수 유형	설명
Vehicle Class	범주형	차량의 크기 및 용도에 따른 차급 정보
Engine Size (L)	연속형	엔진 배기량(리터 단위)
Cylinders	이산형	엔진 실린더 개수
Transmission	범주형	변속기 유형 및 구동 전달 방식
Fuel Consumption City (L/100 km)	연속형	도심 주행 시 연료 소비량
Fuel Consumption Hwy (L/100 km)	연속형	고속도로 주행 시 연료 소비량
CO2 Emissions (g/km)	연속형	차량의 이산화탄소 배출량
Fuel Type	범주형 (종속변수)	차량 연료 종류(휘발유, 디젤 등)

3) 변수 선정 이유

변수명	변수 선정 이유
Vehicle Class	차량의 크기 및 용도를 나타내는 변수로, 차량 중량과 주행 특성을 간접적으로 반영한다. 일반적으로 차급이 커질수록 더 높은 동력과 연료 소비가 요구되므로 연료 종류에 따른 특성이 반영될 가능성이 있다.
Engine Size (L)	엔진 배기량은 차량의 출력과 연료 소비 특성에 직접적인 영향을 미친다. 디젤 차량과 휘발유 차량은 엔진 구조 및 배기량 분포에서 차이를 보일 수 있으므로 중요한 설명변수로 선정하였다.
Cylinders	실린더 수는 엔진의 구조적 특성과 동력 성능을 반영한다. 일반적으로 실린더 수가 증가할수록 엔진 출력과 연료 소비량이 증가하는 경향이 있으며, 연료 종류별 엔진 설계 차이를 설명할 수 있다.
Transmission	변속기 유형은 엔진 동력이 구동계로 전달되는 방식을 의미하며, 연비 및 주행 효율과 밀접한 관련이 있다. 자동변속기, 수동변속기, CVT 등의 차이는 연료 소비 특성에 영향을 줄 수 있다.
Fuel Consumption City (L/100 km)	도심 주행 환경에서의 연료 소비 패턴을 나타내는 변수이다. 차량의 연비 특성과 연료 효율 차이를 반영할 수 있어 연료 종류 분류에 유의미한 정보를 제공한다.
Fuel Consumption Hwy (L/100 km)	고속도로 주행 시 연료 소비 특성을 나타내는 변수이다. 디젤 차량은 장거리·고속 주행에서 상대적으로 높은 효율을 보이는 경우가 많아 연료 종류 구분에 활용 가능하다.
CO2 Emissions (g/km)	연료 연소 과정에서 발생하는 이산화탄소 배출 특성을 반영하는 변수이다. 연료 종류에 따라 연소 효율 및 배출 패턴 차이가 존재하므로 분류에 중요한 변수로 판단하였다.
Fuel Type	본 연구의 종속변수로, 차량의 연료 종류(휘발유, 디젤 등)를 의미한다. 다양한 차량 특성을 이용하여 해당 연료 종류를 분류하는 것을 목표로 한다.

나. 데이터 정제

1) 결측치 제거

결측치는 존재하지 않는 것으로 확인했다.

```

None
Make                                0
Model                              0
Vehicle Class                      0
Engine Size(L)                    0
Cylinders                         0
Transmission                      0
Fuel Type                         0
Fuel Consumption City (L/100 km)  0
Fuel Consumption Hwy (L/100 km)  0
Fuel Consumption Comb (L/100 km)  0
Fuel Consumption Comb (mpg)       0
CO2 Emissions(g/km)              0
dtype: int64

```

2) 중복행 제거

```
중복행 개수 : 2707
```

본 연구에서는 데이터 전처리 과정에서 전체 컬럼 기준으로 완전히 동일한 중복 행이 2707개 존재함을 확인하였다.

완전히 동일한 관측치는 독립적인 데이터로 보기 어렵고, 특정 패턴의 반복을 통해 모델이 해당 샘플을 과도하게 학습하게 만들 수 있다. 이는 데이터 분포를 왜곡하고 모델의 일반화 성능을 저하시킬 가능성이 존재한다.

따라서 데이터의 편향을 방지하고 분석의 신뢰성을 확보하기 위해 모든 컬럼 값이 동일한 중복 행을 제거하였다. 중복행 제거 후 남은 데이터는 4307개다.

```
중복 제거 후 데이터 크기 : (4307, 7)
```

```

Fuel Type
X    2353
Z    1846
D     108
Name: count, dtype: int64

```

3) 이상치제거

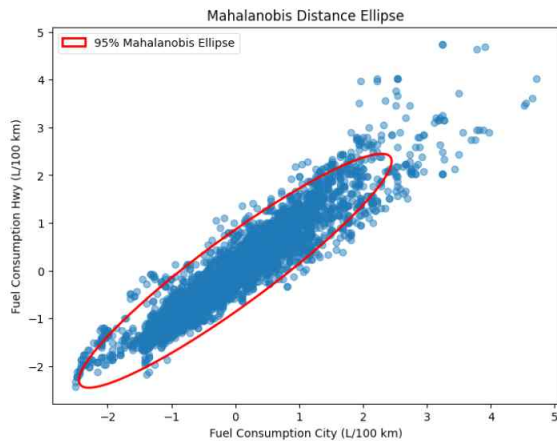
가) Scaling (표준화)

마할라노비스를 하기 전 단위를 갖는 변수들을 표준화하였다.

나) mahalanobis distance 기반 이상치 검출

단일 변수 기준 뿐만 아니라 다변량 구조를 반영하기 위해 마할라노비스 거리 (Mahalanobis Distance)를 활용하여 이상치를 탐지하였다.

마할라노비스 거리는 공분산 구조를 반영하여 데이터가 평균으로부터 얼마나 벗어나 있는지를 측정하는 지표로, 변수 간 상관관계를 고려한 이상치 탐지가 가능하다.



```

사용 변수 개수: 5
95% 기준값: 11.070
99% 기준값: 15.086

95% 기준 이상치 개수:
496

99% 기준 이상치 개수:
317
  
```

분석 결과, 통계적 임계치를 초과하는 496개의 관측치가 식별되었다.

나) 이상치 처리 전략

일반적으로 이상치는 제거 대상이 될 수 있으나, 본 연구에서는 이를 단순한 노이즈가 아닌 구조적으로 의미 있는 데이터로 판단하여 제거하지 않고 유지하였다.

일반 차량은 엔진 크기와 탄소 배출량 간의 선형적 관계를 따르는 반면, 일부 차량은 이러한 공분산 구조에서 벗어난 특성을 가진다. 예를 들어, 고성능 차량은 기존의 선형 관계를 따르지 않는 경우가 존재한다.

이러한 데이터는 단순한 오류가 아니라 실제 시장에서 존재하는 다양한 차량 특성을 반영하며, 이를 제거할 경우 모델이 현실적인 분포를 충분히 학습하지 못할 가능성이 있다.

또한 정보 이론적 관점에서 이러한 관측치는 발생 확률이 낮은 대신 높은 정보량을 가지므로, 모델의 표현력을 높이기 위해 유지하는 것이 타당하다고 판단하였다.

다) 최종 판단

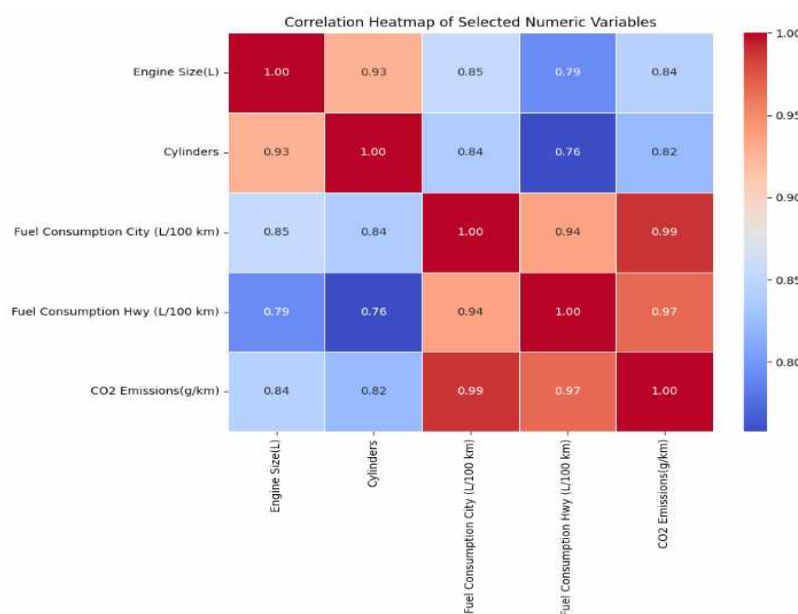
따라서 본 연구에서는 마할라노비스 거리 기준으로 식별된 이상치를 제거하지 않고 유지함으로써, 데이터의 정보 손실을 최소화하고 모델의 강건성을 확보하고자 하였다.

다.회귀 가정 진단

1) 다중공선성

가) VIF 분석

	Variable	VIF
4	CO2 Emissions(g/km)	108.606
3	Fuel Consumption City (L/100 km)	58.844
2	Fuel Consumption Hwy (L/100 km)	20.847
0	Engine Size(L)	8.308
1	Cylinders	7.688



설명변수들 간 높은 다중공선성이 확인되었다. 자동차 데이터의 특성상 Engine Size, Cylinders, Fuel Consumption, CO2 Emissions 변수들은 차량의 출력 및 연료 소비 특성과 직접적으로 연관되어 있으므로 서로 높은 상관관계를 가질 가능성이 높다. 일반적으로 이러한 다중공선성은 회귀계수 추정의 불안정성과 분산 증가 문제를 유발하며, 변수 해석의 신뢰성을 저하시킬 수 있다. 실제로 Vatcheva et al.(2016)은 시뮬레이션 연구를 통해 다중공선성이 회귀계수의 표준오차 증가와 계수 부호 변화(sign reversal)를 유발할 수 있음을 보였다. 그러나 동일 연구에서는 “Multicollinearity does not affect the overall fit or the predictions of the model” 이라고 설명하며, 다중공선성이 반드시 모델의 예측 성능 저하로 이어지는 것은 아님을 언급하였다.

또한 Lieberman and Morris(2014)는 매우 높은 수준의 다중공선성이 존재하더라도 예측 정확도 자체는 크게 감소하지 않을 수 있음을 보고하였으며, 특히 “Even very extreme multicollinearity did not affect OLS or LR prediction accuracy” 라고 설명하였다. 이는 다중공선성이 변수 해석에는 문제를 유발할 수 있으나, 예측 중심 모델의 성능에는 반드시 치명적인 영향을 주지 않을 수 있음을 시사한다.

본 연구에서는 다중공선성 완화를 위해 PCA 기반 차원축소를 적용하여 주성분 변수만을 사용하는 방법을 검토하였다. 그러나 PCA 적용 이후 F1-score가 오히려 감소하는 결과가 나타났다. 이는 PCA가 전체 분산을 최대한 보존하는 방향으로 변환을 수행하지만, 분류에 중요한 클래스 분리 정보(class separation information)까지 반드시 유지하는 것은 아니기 때문으로 해석된다. 특히 본 데이터에서는 연료 종류에 따른 차이가 원래 변수들의 특정 조합 및 구조적 관계에서 나타났을 가능성이 있으며, PCA 과정에서 이러한 정보 일부가 손실되면서 분류 성능이 감소한 것으로 판단된다.

또한 본 연구의 목적은 개별 변수의 회귀계수를 해석하는 것이 아니라 연료 유형을 정확하게 분류하는 데 있었다. 따라서 변수 해석 안정정보보다 실제 분류 성능(F1-score)을 우선적으로 고려하였으며, PCA 적용 전후의 성능 비교 결과를 바탕으로 최종적으로 원변수를 유지한 상태에서 LDA, QDA, SVM 분류모형을 적용하였다. 이는 공선성 자체를 무시한 것이 아니라, 공선성 완화 기법 적용 이후의 실제 성능 변화를 종합적으로 검토한 결과에 기반한 선택이다.

The Precise Effect of Multicollinearity on Classification Prediction

Mary G. Lieberman

John D. Morris

Florida Atlantic University

The results of Morris and Lieberman (2012) were extended to include more extreme multicollinearity conditions, with specific attention paid to the effect of a wide range of multicollinearity levels on Ordinary Least Squares and Logistic Regression cross-validated classification accuracy, and the attendant increase in prediction accuracy afforded by Ridge Regression and Principal Components models with increasing validity concentration. Even very extreme multicollinearity did not affect OLS or LR prediction accuracy, but, given attendant validity concentration, did enable Ridge Regression and Principal Components to exceed the accuracy of these typically used classification techniques. Clarification regarding how these results bear on admonitions regarding multicollinearity was tendered.



HHS Public Access

Author manuscript

Epidemiology (Sunnyvale). Author manuscript; available in PMC 2016 June 01.

Published in final edited form as:

Epidemiology (Sunnyvale). 2016 April ; 6(2): . doi:10.4172/2161-1165.1000227.

Multicollinearity in Regression Analyses Conducted in Epidemiologic Studies

Multicollinearity arises when at least two highly correlated predictors are assessed simultaneously in a regression model. The adverse impact of multicollinearity in regression analysis is very well recognized and much attention to its effect is documented in the literature [1-11]. The statistical literature emphasizes that the main problem associated with multicollinearity includes unstable and biased standard errors leading to very unstable p-values for assessing the statistical significance of predictors, which could result in unrealistic and untenable interpretations [4, 7, 12, 13]. Multicollinearity does not affect the overall fit or the predictions of the model [14]. If the purpose of the regression model is to investigate associations, multicollinearity among the predictor variables can obscure the computation and identification of key independent effects of collinear predictor variables on the outcome

다) 최종 변수 선정

1) 설명변수 그룹화

Transmission변수는 AS4, AS6, AS8, AV, M5, M6 등 변속기 유형이 과도하게 세분화되어 있었다. 이에 따라 변수의 복잡도를 줄이고 해석의 용이성을 높이기 위해 변속기 방식을 기준으로 AS, A, M, AV의 4개 범주로 재분류하였다.

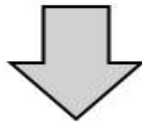
```
Transmission_Group
AS      1968
A       1216
M        714
AV       409
Name: count, dtype: int64
```

2) 반응변수 재정의 (Labeling):

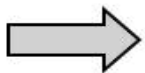
Fuel Type 변수에서 연구 목적에 맞게 연료 종류 중 디젤(D)과 휘발유(일반/고급) 데이터만을 추출하여 사용하였다.

단순히 디젤과 휘발유로 구분하지 않고, 디젤과 일반휘발유, 고급휘발유로 나눈 이유는 평균의 함정에 빠지지 않기 위함이다.

구분	n	Engine Size 평균	Cylinders 평균	Comb 연료소비	CO ₂ 평균
Diesel	147	2.533	4.925	8.750	235.238
Gasoline	5,804	3.122	5.583	10.739	250.152



구분	n	Engine Size 평균	Cylinders 평균	Comb 연료소비	CO ₂ 평균
Diesel	147	2.533	4.925	8.750	235.238
Regular Gasoline	3,039	2.837	5.040	10.126	235.978
Premium Gasoline	2,765	3.435	6.179	11.413	265.731



겉으로 보면 디젤의 이산화탄소 배출량이 적어 보이지만 이를 세분화하면 정보 손실이 발생하고 있음을 알 수 있음

디젤과 휘발유로만 단순 분류할 경우 CO₂ 배출량 평균 차이가 약 15 정도로 나타났으나, 디젤·일반휘발유·고급휘발유로 세분화하여 비교하면 일반휘발유는 디젤과 유사한 CO₂ 배출 특성을 보였다. 따라서 평균에 의해 가려질 수 있는 정보 손실을 방지하기 위해 반응변수를 3개의 클래스로 구분하였다.

라) K-Fold 교차검증 기반 학습·평가 데이터 구성

분류모델의 성능을 보다 안정적으로 평가하기 위해 K-Fold 교차검증(k=5)을 사용하였다. K-Fold 교차검증은 데이터를 여러 번 나누어 학습과 검증을 반복하므로, 특정 데이터 분할에 따른 성능 편향을 줄이고 모델의 일반화 성능을 보다 신뢰성 있게 평가할 수 있다는 장점이 있다. 본 분석에서는 각 Fold마다 Train/Test 데이터를 7:3 비율로 구성하여 모델 성능을 평가하였다.

```
X_train: (3445, 12)
X_test : (862, 12)
y_train: (3445,)
y_test : (862,)

Train class distribution
Fuel Type
X      1882
Z      1477
D        86
Name: count, dtype: int64

Test class distribution
Fuel Type
X      471
Z      369
D       22
Name: count, dtype: int64

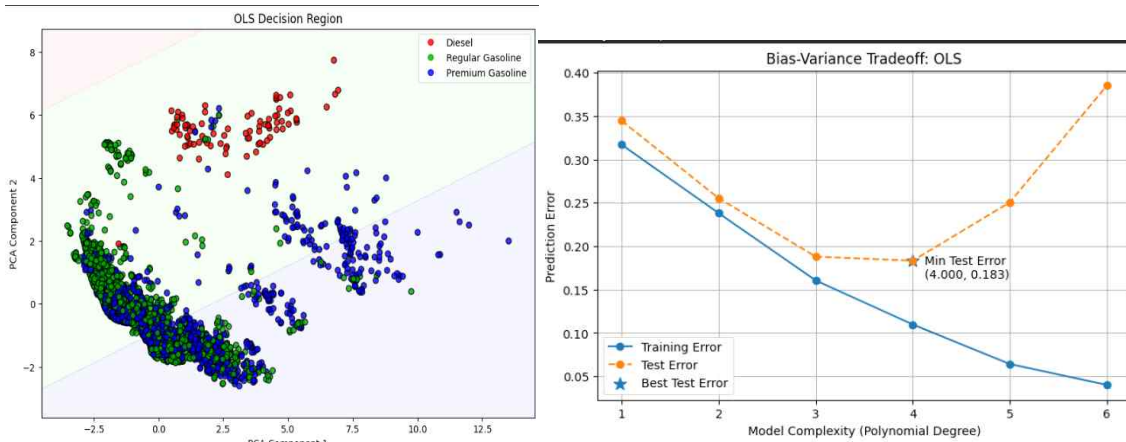
✅ Train/Test split + Scaling + Stratified K-Fold 설정 완료
```

1. 여러 모델 비교

1-1. OLS(Ordinary Least Squares)

가장 기초적인 선형 모형인 OLS(선형 확률 모형)를 적용하여 연료 종류 분류를 시도하였다. OLS 결과, 디젤 차량과 일반휘발류, 고급휘발류 차량의 비율이 약 20:20:1 수준으로 매우 불균형한 본 데이터셋의 특징으로 인해 모델이 다수 클래스인 휘발유에 편향되는 문제가 발생함을 확인하였다.

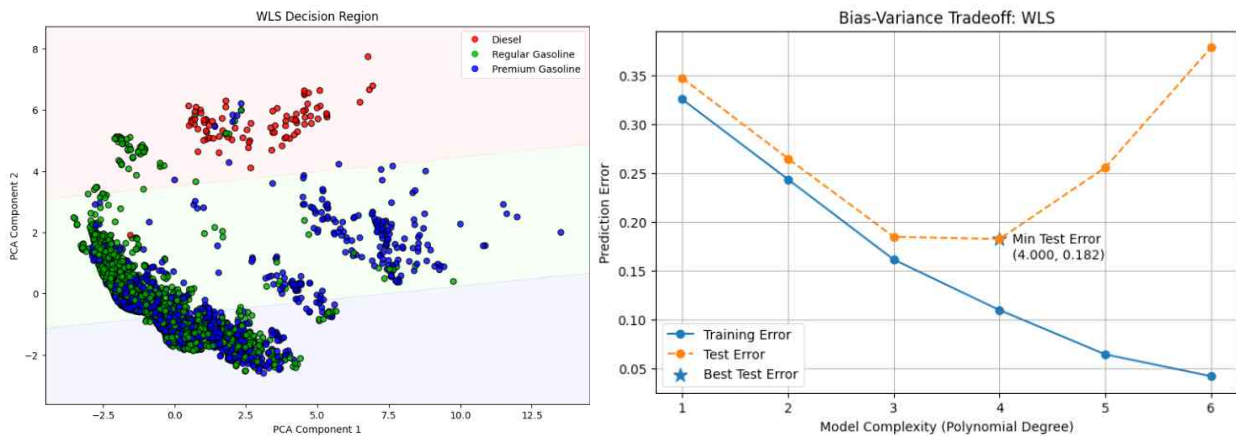
특히 디젤 차량을 거의 식별하지 못하는 현상이 나타났으며, 이는 단순 OLS가 데이터 불균형 상황에서 소수 클래스를 충분히 반영하지 못함을 보여준다.



1-2. WLS(Weighted Least Squares)

본 데이터셋은 디젤 데이터와 일반휘발류, 고급휘발류의 데이터 크기가 20:20:1 수준으로 데이터 크기의 불균형 구조를 가진다. 이러한 경우 일반적인 OLS는 다수 클래스에 지나치게 최적화되어 소수 클래스인 디젤 차량을 제대로 분류하지 못할 가능성이 높다. 이를 보완하기 위해 본 연구에서는 WLS(가중최소제곱법)를 적용하였다. WLS는 손실 함수 내에서 특정 클래스에 더 높은 가중치를 부여할 수 있으며, 본 분석에서는 디젤 클래스에 20배의 가중치를 부여하였다. 여러 가중치를 비교한 결과, 20배 가중치가 가장 우수한 분류 성능을 보였다.

이를 통해 전체 정확도 일부는 감소하더라도, 디젤 차량에 대한 Recall을 높이는 방향으로 모델을 개선하고자 하였다.



1-3 OLS와 WLS 성능 비교

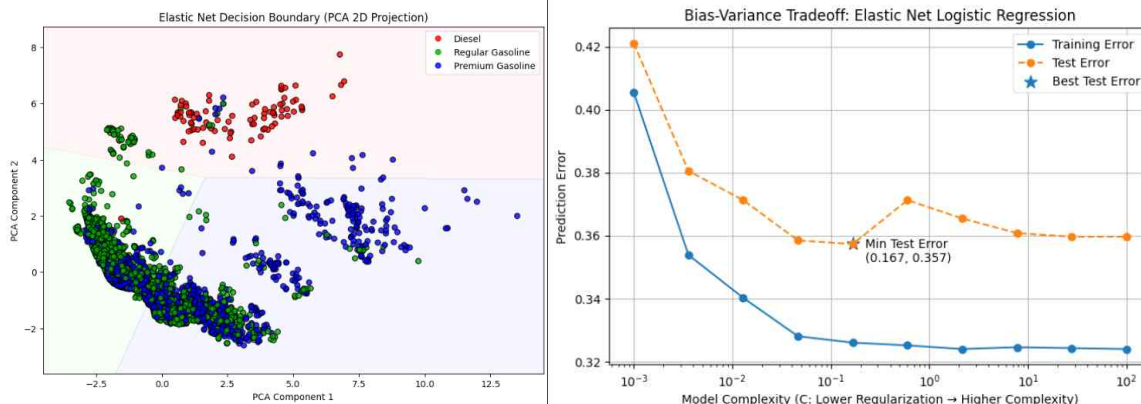
	Model	Accuracy	Precision_macro	Recall_macro	F1_macro
0	OLS	0.655452	0.762316	0.723361	0.736774
1	WLS	0.653132	0.760169	0.749463	0.748286

1-4. ELASTIC NET

OLS 및 WLS 기반 분류를 수행한 결과, 데이터의 클래스 불균형과 변수 간 높은 상관성으로 인해 일부 클래스에서 분류 성능이 충분히 안정적으로 나타나지 않았다. 이에 따라 과적합을 완화하고 변수 선택 및 계수 안정성을 동시에 고려할 수 있는 Elastic Net 모델을 추가적으로 적용해보았다.

	Model	Accuracy	Precision_macro	Recall_macro	F1_macro
0	ElasticNet_Logistic	0.654292	0.761153	0.750171	0.748976

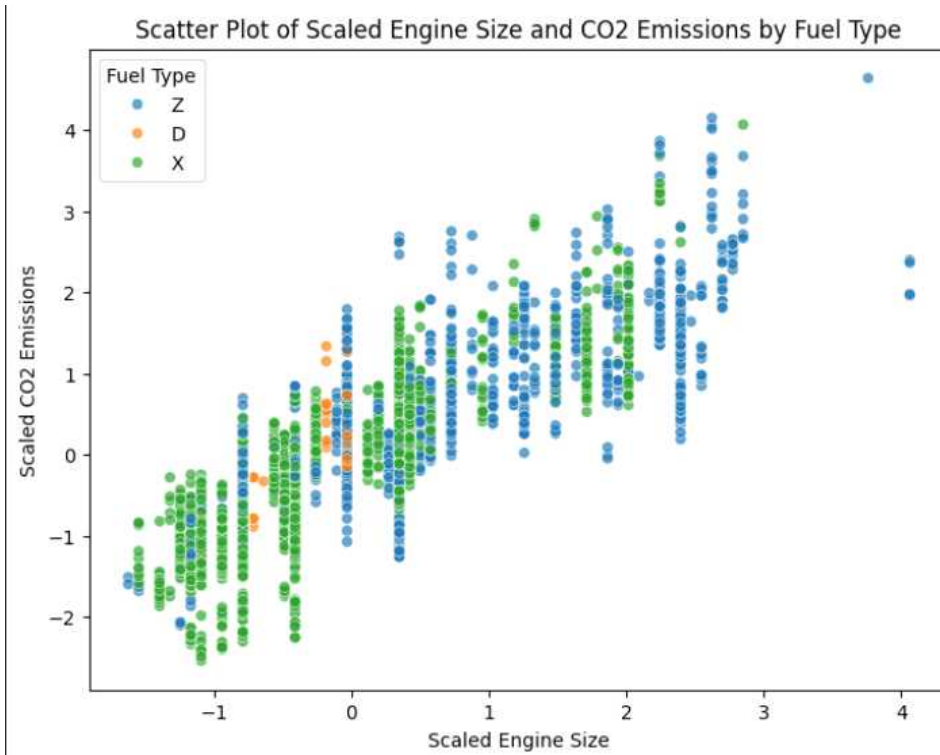
	Engine type	Engine Size(L)	Cylinders	Fuel Consumption City (L/100 km)	Fuel Consumption Hwy (L/100 km)	CO2 Emissions(g/km)	Mahalanobis Distance	Mahalanobis Distance Squared	Outlier 95	Outlier 99	Transmission Group AS	Transmission Group AV	Transmission Group_M
0	11	1100	75	57	00	00	0.77100	0.78800	-0.55741	1.22760	0.181638	-1.038836	0.000000
1	24	1675	49	11	73	96	0.00000	1.98500	0.00000	-0.10680	-0.02800	0.220259	0.038785
2	17	1722	25	34	00	85	-1.53510	0.25530	0.31890	-0.17690	0.00000	0.00000	0.000000



1-3. 선형 모델의 한계

OLS와 WLS, ELASTIC NET은 기본적으로 변수 간의 선형 관계를 기반으로 분류를 수행한다. 그러나 데이터의 실제 분포를 시각적으로 확인한 결과, 단순 선형 경계만으로는 두 클래스를 충분히 구분하기 어렵다고 판단하였다.

이를 확인하기 위해 스케일링된 Engine Size와 CO2 Emissions 변수에 대해 연료 종류별 산점도를 시각화하였다.



- 휘발유(Petrol) 데이터는 엔진 크기 전 영역에 걸쳐 넓은 분산을 보이며 데이터가 광범위하게 퍼져 있었다.
- 반면 디젤(Diesel) 데이터는 표본 수 자체가 적고 특정 구간에 밀집되어 있는 형태를 보였다.

이는 두 클래스 간 공분산 구조가 서로 상이할 가능성을 시사한다.

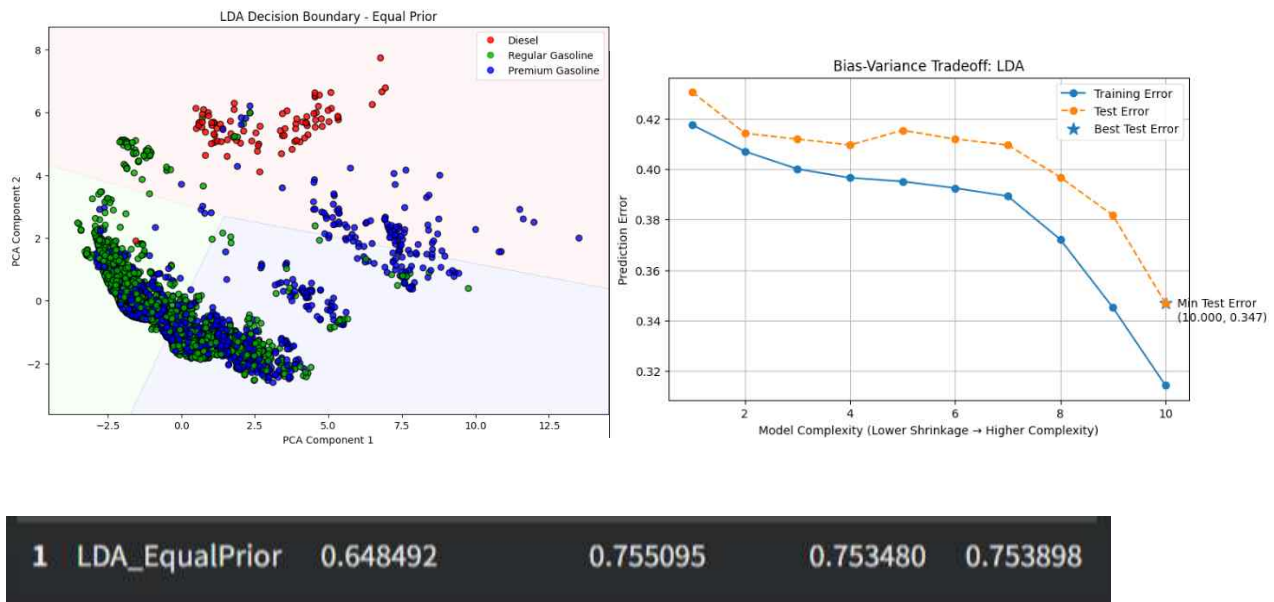
6.균형 데이터에서의 딜레마

- 디젤 데이터의 비율이 약 1:20:20 수준으로 매우 낮아, 디젤 클래스의 실제 분산 구조를 안정적으로 추정하기 어려웠다.
- 이러한 상황에서 QDA를 적용할 경우, 적은 수의 디젤 데이터에 모델이 지나치게 적합되어 새로운 데이터에 대한 일반화 성능이 저하될 가능성이 존재하였다.

즉, 시각적으로는 두 클래스의 분포 형태가 상이하여 QDA가 이론적으로 적합해 보였으나, 디젤 데이터의 희소성으로 인해 과적합(Overfitting) 위험 역시 함께 존재한다고 판단하였다.

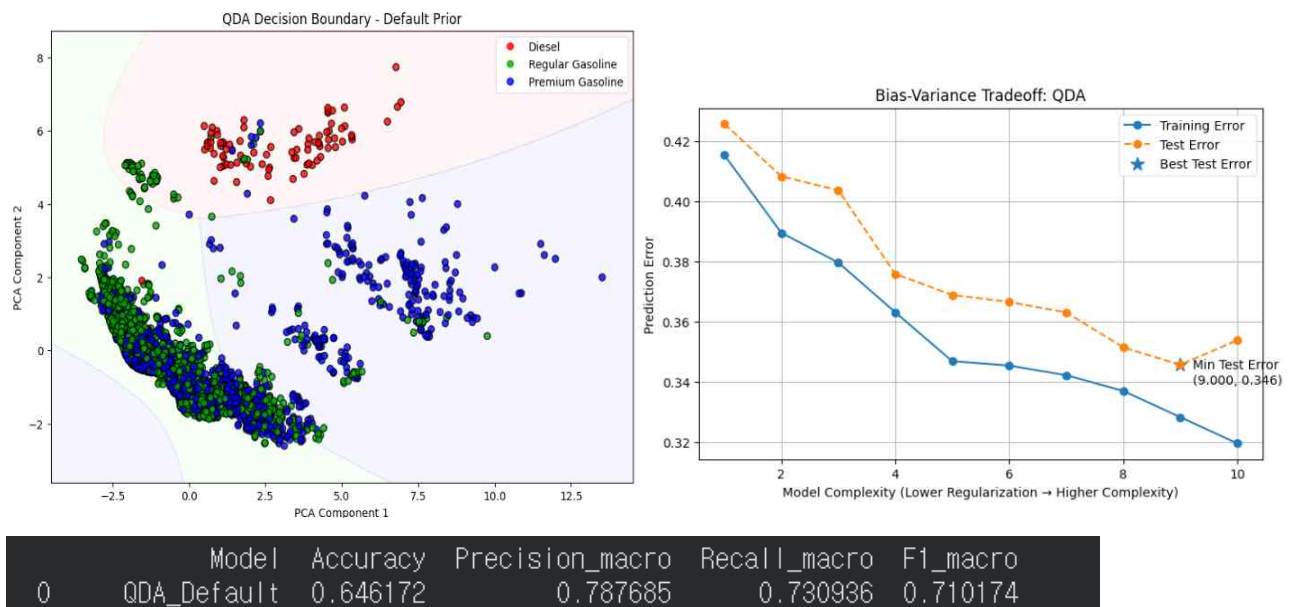
이에 따라 본 연구에서는 LDA와 QDA를 모두 적용한 뒤, K-Fold 교차검증을 통해 두 모델의 성능을 비교하였다. 최종적으로는 편향(Bias)-분산(Variance) Trade-off를 함께 고려하여 모델의 적합성을 평가하고자 하였다.

1-4. LDA



1 LDA_EqualPrior 0.648492 0.755095 0.753480 0.753898

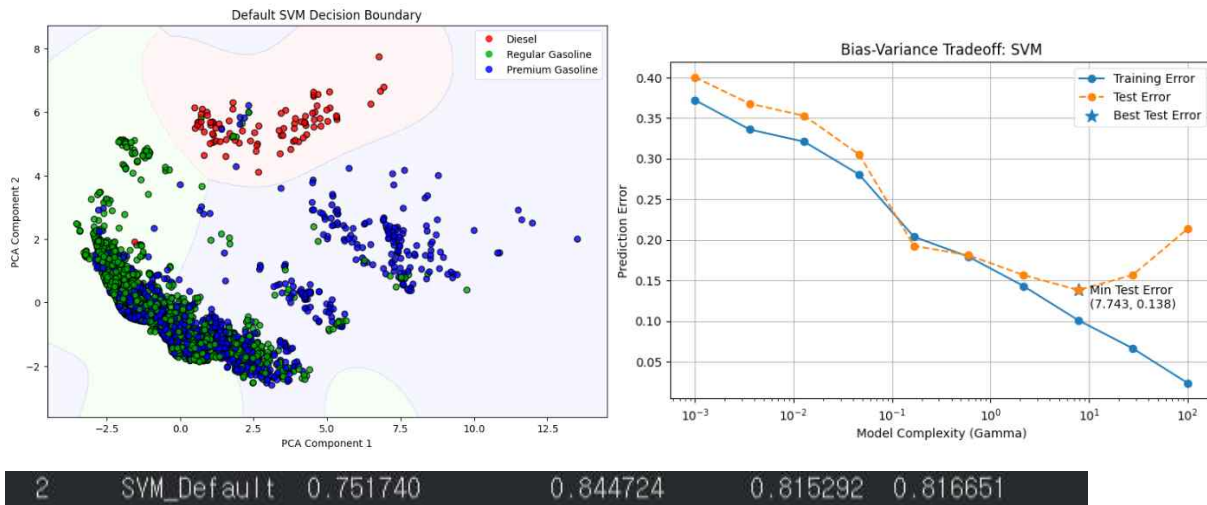
1-5. QDA(Quadratic Discriminant Analysis)



Model	Accuracy	Precision_macro	Recall_macro	F1_macro
0 QDA_Default	0.646172	0.787685	0.730936	0.710174

K-Fold 교차검증 결과, QDA는 디젤 데이터의 희소성과 변수 간 높은 공선성으로 인해 공분산행렬 추정치가 불안정해지는 문제가 존재하였다. 반면 LDA는 공통 공분산행렬을 가정하는 보다 단순한 구조를 사용하여 안정적인 분류 성능을 보였으며, 평균적인 평가 지표 또한 더 우수하게 나타났다.

1-6. SVM(Support Vector Machine)



앞선 실험에서 LDA를 적용한 결과, 선형 모델 대비 분류 성능 향상이 확인되었다. 이는 본 데이터셋에서 중요한 요소가 단순 선형 분리가 아니라 비선형 경계의 형성에 있음을 시사한다.

이에 본 연구에서는 비선형 경계의 가능성을 확장하되, LDA의 한계인 정규분포 가정을 극복하기 위해 비선형 SVM(Kernel SVM)을 최종 모델 후보로 선정하였다.

SVM은 데이터의 확률분포 자체를 추정하지 않고, 클래스 간 경계를 최대화하는 방식으로 동작한다. 특히 Support Vector만을 중심으로 결정경계를 형성하기 때문에 데이터 중첩이 심한 영역에서도 보다 안정적이고 강건한(Robust) 성능을 기대할 수 있다. 또한 RBF Kernel과 같은 커널 기법을 활용하면 저차원 공간에서 선형적으로 분리되지 않는 데이터를 고차원 공간으로 매핑하여 더욱 복잡한 비선형 경계를 형성할 수 있다. 본 연구에서는 불균형 데이터 문제를 완화하기 위해 클래스 가중치(class weight)를 함께 적용하였으며, 이를 통해 클래스 불균형과 데이터 중첩 문제를 동시에 해결하고자 하였다.

2. 모델 별 성능 분석 및 Bias-Variance Trade-off

OLS는 가장 단순한 선형 모델로 높은 Bias와 낮은 Variance를 가지는 특성이 있다. 그러나 데이터 불균형 상황에서 다수 클래스에 편향되며, 디젤 차량을 거의 탐지하지 못하는 문제가 발생하였다.

WLS는 소수 클래스에 가중치를 부여함으로써 디젤 차량에 대한 Recall을 개선하였다. 다만 여전히 선형 경계 기반 모델이라는 한계가 존재하였다.

QDA는 클래스별 서로 다른 공분산 구조를 반영함으로써 비선형 경계를 형성할 수 있었으며, 선형 모델보다 향상된 분류 성능을 보였다. 그러나 디젤 데이터의 수가 매우 적은 1:20:20 불균형 구조에서는 클래스별 공분산 행렬 추정이 불안정해질 수 있으며,

이로 인해 과적합(Overfitting) 위험이 존재하였다.

즉, QDA는 Bias는 감소하지만 Variance가 증가하는 특성을 보였다.

반면 SVM은 데이터의 분포를 직접 추정하지 않고 결정경계를 최대화하는 방식으로 동작한다. 특히 Support Vector를 중심으로 경계를 학습하기 때문에 데이터 중첩 상황에서도 안정적인 성능을 보였으며, Kernel 기법을 통해 복잡한 비선형 경계를 효과적으로 학습할 수 있었다.

또한 클래스 가중치를 적용함으로써 불균형 데이터 환경에서도 디젤 차량 탐지 성능을 안정적으로 유지할 수 있었다.

3. 성능 평가 및 비교

본 데이터셋은 디젤 차량 비율이 매우 낮은 불균형 구조를 가진다. 따라서 단순 Accuracy만으로는 모델 성능을 적절히 평가하기 어렵다.

특히 본 연구에서는 디젤 차량을 휘발유 차량으로 잘못 분류하는 경우의 비용이 크다고 판단하였다. 즉, 디젤 차량을 놓치지 않는 능력(Recall)을 중요하게 고려하였다.

이에 따라 성능이 가장 좋은 SVM 모델의 성능을 다음 지표들을 함께 활용하여 다시 평가해보았다.

- F1 Score: Precision과 Recall의 균형 평가
- F2 Score: Recall에 더 큰 가중치를 부여한 지표

F1.5-score: 0.714

F2-score: 0.7257

4. 결론

비선형 SVM은 전체적인 분류 균형과 디젤 차량 탐지 성능을 동시에 만족하는 가장 안정적인 모델로 나타났다.

특히 클래스 불균형 상황에서도 디젤 차량에 대한 높은 Recall을 유지하였으며, 데이터 중첩이 심한 영역에서도 강건한 성능을 보였다.

따라서 본 연구에서는 클래스 불균형 문제와 비선형 경계 문제를 동시에 효과적으로 해결할 수 있는 모델로 비선형 SVM을 최종 최적 모델로 선정하였다.

```

# 결측치(빈 값) 개수 확인
print(df.isnull().sum())

# =====
# 필요한 변수만 선택
# =====

selected_columns = [
    'Engine Size(L)',
    'Cylinders',
    'Transmission',
    'Fuel Consumption City (L/100 km)',
    'Fuel Consumption Hwy (L/100 km)',
    'CO2 Emissions(g/km)',
    'Fuel Type'
]

df = df[selected_columns]

print("✅ 변수 선택 완료")
print(df.head())

# =====
# 사용할 연료 종류만 선택
# X : Regular Gasoline
# Z : Premium Gasoline
# D : Diesel
# =====

df = df[df['Fuel Type'].isin(['X', 'Z', 'D'])]

print("✅ Fuel Type 필터링 완료")
print(df['Fuel Type'].value_counts())

# =====
# 중복행 개수 확인
# =====

duplicate_count = df.duplicated().sum()

print("중복행 개수 :", duplicate_count)

# =====
# 중복행 제거
# =====

df = df.drop_duplicates()

print("중복 제거 후 데이터 크기 :", df.shape)

print(df['Fuel Type'].value_counts())

```

```
import matplotlib.pyplot as plt
import seaborn as sns

# 사용할 수치형 변수

numeric_columns = [
    'Engine Size(L)',
    'Cylinders',
    'Fuel Consumption City (L/100 km)',
    'Fuel Consumption Hwy (L/100 km)',
    'CO2 Emissions(g/km)'
]

# Boxplot 그리기
# Fuel Type별 분포 확인

for col in numeric_columns:

    plt.figure(figsize=(8, 5))

    sns.boxplot(
        x='Fuel Type',
        y=col,
        data=df
    )

    plt.title(f'{col} by Fuel Type')
    plt.xlabel('Fuel Type')
    plt.ylabel(col)

    plt.show()
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import chi2
from sklearn.preprocessing import StandardScaler

# =====
# 1. Mahalanobis distance에 사용할 변수들 변수
# =====

numeric_cols = [
    'Engine Size(L)',
    'Cylinders',
    'Fuel Consumption City (L/100 km)',
    'Fuel Consumption Hwy (L/100 km)',
    'CO2 Emissions(g/km)'
]

X = df[numeric_cols].copy()

# =====
# 2. 표준화
# =====

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# =====
# 3. 평균변인, 공분산행렬, 역공분산행렬 계산
# =====

mean_vec = np.mean(X_scaled, axis=0)
cov_mat = np.cov(X_scaled, rowvar=False)
inv_cov_mat = np.linalg.pinv(cov_mat) # 공분산 대역행렬 pinv 사용

# =====
# 4. Mahalanobis distance 계산
# =====

diff = X_scaled - mean_vec

md_squared = np.sum(diff @ inv_cov_mat * diff, axis=1)
md = np.sqrt(md_squared)

df['Mahalanobis_Distance'] = md
df['Mahalanobis_Distance_Squared'] = md_squared

# =====
# 5. 카이제르 기준값 설정
# 카이제르 = 변수 개수
# =====

p = len(numeric_cols)
threshold_95 = chi2.ppf(0.95, df=p)
threshold_99 = chi2.ppf(0.99, df=p)

df['Outlier_95'] = df['Mahalanobis_Distance_Squared'] > threshold_95
df['Outlier_99'] = df['Mahalanobis_Distance_Squared'] > threshold_99

print("사용 변수 개수:", p)
print(f"95% 기준값: {threshold_95:.3f}")
print(f"99% 기준값: {threshold_99:.3f}")

print("\n95% 기준 이상치 개수:")
print(df['Outlier_95'].sum())

print("\n99% 기준 이상치 개수:")
print(df['Outlier_99'].sum())

```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from scipy.stats import chi2
from matplotlib.patches import Ellipse

# =====
# 변수 2개 선택
# =====

feature_names = [
    'Fuel Consumption City (L/100 km)',
    'Fuel Consumption Hwy (L/100 km)'
]

X = df[feature_names].values

# =====
# 표준화
# =====

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# =====
# 평균, 공분산 계산
# =====

mean = np.mean(X_scaled, axis=0)
cov = np.cov(X_scaled, rowvar=False)

# =====
# 고유값, 고유벡터 계산
# =====

eigenvalues, eigenvectors = np.linalg.eigh(cov)

# 큰 고유값부터 정렬
order = eigenvalues.argsort()[::-1]

eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]

# =====
# 카이제르 기반 ellipse 크기
# 95% confidence ellipse
# =====

chi2_val = chi2.ppf(0.95, df=2)

width = 2 * np.sqrt(eigenvalues[0] * chi2_val)
height = 2 * np.sqrt(eigenvalues[1] * chi2_val)

# 방향도 계산
angle = np.degrees(
    np.arctan2(*eigenvectors[:,0][::-1])
)

```

```

# =====
# 시판용
# =====

fig, ax = plt.subplots(figsize=(8, 6))

# 데이터
ax.scatter(
    X_scaled[:,0],
    X_scaled[:,1],
    alpha=0.5
)

# Mahalanobis ellipse
ellipse = Ellipse(
    xy=mean,
    width=width,
    height=height,
    angle=angle,
    edgecolor='red',
    facecolor='none',
    linewidth=2,
    label='95% Mahalanobis Ellipse'
)

ax.add_patch(ellipse)

ax.set_title('Mahalanobis Distance Ellipse')
ax.set_xlabel(feature_names[0])
ax.set_ylabel(feature_names[1])

ax.legend()

plt.show()

```

Figure 1: Mahalanobis Distance Ellipse

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor

numeric_cols = [
    "Engine Size(L)",
    "Cylinders",
    "Fuel Consumption Hwy (L/100 km)",
    "Fuel Consumption City (L/100 km)",
    "CO2 Emissions(g/km)"
]

df_vif_num = df[numeric_cols].copy()

for col in numeric_cols:
    df_vif_num[col] = pd.to_numeric(df_vif_num[col], errors="coerce")

df_vif_num = df_vif_num.dropna().copy()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_vif_num)

vif_numeric = pd.DataFrame({
    "Variable": numeric_cols,
    "VIF": [
        variance_inflation_factor(X_scaled, i)
        for i in range(X_scaled.shape[1])
    ]
})

vif_numeric = vif_numeric.sort_values("VIF", ascending=False)

display(vif_numeric.round(3))

```



```

import matplotlib.pyplot as plt
import seaborn as sns

# =====
# 선택된 수치형 변수만 사용
# =====

numeric_cols = [
    'Engine Size(L)',
    'Cylinders',
    'Fuel Consumption City (L/100 km)',
    'Fuel Consumption Hwy (L/100 km)',
    'CO2 Emissions(g/km)'
]

corr_matrix = df[numeric_cols].corr()

# =====
# 상관관계 Heatmap
# =====

plt.figure(figsize=(9, 7))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title('Correlation Heatmap of Selected Numeric Variables')
plt.show()

```

```

# =====
# Transmission 그룹화
# =====

def transmission_group(x):

    if x.startswith('AS'):
        return 'AS'

    elif x.startswith('AV'):
        return 'AV'

    elif x.startswith('A'):
        return 'A'

    elif x.startswith('M'):
        return 'M'

    else:
        return 'Other'

# 그룹화 변수 생성
df['Transmission_Group'] = df['Transmission'].apply(transmission_group)

print(f"변수: 그룹화 완료")

print(df['Transmission_Group'].value_counts())

```



```

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler

# =====
# 1. X, y 분리
# =====

X = df.drop(columns=['Fuel Type'])
y = df['Fuel Type']

# =====
# 2. Train / Test split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)

print("\nTrain class distribution")
print(y_train.value_counts())

print("\nTest class distribution")
print(y_test.value_counts())

# =====
# 3. Scaling
# =====

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# =====
# 4. Stratified K-Fold CV
# =====

skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

print("\n✅ Train/Test split + Scaling + Stratified K-Fold 📊 📈 📉")

```

```

import numpy as np
import pandas as pd

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_validate
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# =====
# 1. y 숫자 인코딩
# =====

fuel_map = {
    'D': 0,
    'X': 1,
    'Z': 2
}

y_train_num = y_train.map(fuel_map)
y_test_num = y_test.map(fuel_map)

# =====
# 2. 예측값을 0,1,2 클래스로 변환하는 함수
# =====

def to_class(y_pred_cont):
    y_pred = np.round(y_pred_cont).astype(int)
    y_pred = np.clip(y_pred, 0, 2)
    return y_pred

# =====
# 3. OLS
# =====

ols = LinearRegression()

ols.fit(X_train_scaled, y_train_num)

y_pred_ols_cont = ols.predict(X_test_scaled)
y_pred_ols = to_class(y_pred_ols_cont)

# =====
# 4. WLS
# Diesel(D=0)에 큰 가중치 부여
# =====

sample_weight = np.ones(len(y_train_num))

# D가 아닌 즉 경유 또는 D에 가중치 부여
sample_weight[y_train_num == 0] = 40

wls = LinearRegression()

wls.fit(
    X_train_scaled,
    y_train_num,
    sample_weight=sample_weight
)

y_pred_wls_cont = wls.predict(X_test_scaled)
y_pred_wls = to_class(y_pred_wls_cont)

```

```

import matplotlib.pyplot as plt
import seaborn as sns

# =====
# 선택된 수치형 변수만 사용
# =====

numeric_cols = [
    'Engine Size(L)',
    'Cylinders',
    'Fuel Consumption City (L/100 km)',
    'Fuel Consumption Hwy (L/100 km)',
    'CO2 Emissions(g/km)'
]

corr_matrix = df[numeric_cols].corr()

# =====
# 상관관계 Heatmap
# =====

plt.figure(figsize=(9, 7))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title('Correlation Heatmap of Selected Numeric Variables')
plt.show()

```

```

# -----
# 5. OLS
# -----

ols = LinearRegression()

plot_linear_boundary(
    ols,
    'OLS Decision Region'
)

# -----
# 6. WLS
# -----

sample_weight = np.ones(len(y_train_num))

sample_weight[y_train_num == 0] = 40

wls = LinearRegression()

wls.fit(
    X_train_pca,
    y_train_num,
    sample_weight=sample_weight
)

# Grid 📏
x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

grid = np.c_[xx.ravel(), yy.ravel()]

Z_cont = wls.predict(grid)

Z = to_class(Z_cont)

Z = Z.reshape(xx.shape)

cmap_light = ListedColormap([
    '#FFCCCC',
    '#CCFFCC',
    '#CCCCFF'
])

cmap_bold = ListedColormap([
    '#FF0000',
    '#00AA00',
    '#0000FF'
])

```

```
plt.figure(figsize=(10, 7))

plt.contourf(
    xx,
    yy,
    Z,
    alpha=0.3,
    cmap=cmap_light
)

scatter = plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=y_train_num,
    cmap=cmap_bold,
    edgecolor='k',
    alpha=0.8
)

plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')

plt.title('KLS Decision Region')

handles = scatter.legend_elements()[0]

plt.legend(
    handles,
    ['Diesel', 'Regular Gasoline', 'Premium Gasoline']
)

plt.show()
```

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# =====
# 1. 모델 정의
# =====

# 기본 QDA
qda_default = QuadraticDiscriminantAnalysis()

# Prior 준 QDA
qda_weighted = QuadraticDiscriminantAnalysis(
    priors=[1/3, 1/3, 1/3]
)

# 기본 SVM
svm_default = SVC(
    kernel='rbf',
    random_state=42
)

# Class Weight 준 SVM
svm_weighted = SVC(
    kernel='rbf',
    class_weight='balanced',
    random_state=42
)

# =====
# 2. 모델 리스트
# =====

models = {
    'QDA_Default': qda_default,
    'QDA_EqualPrior': qda_weighted,
    'SVM_Default': svm_default,
    'SVM_Balanced': svm_weighted
}

```

```

# =====
# 3. 결과 저장
# =====

results = []

for name, model in models.items():

    # 학습
    model.fit(X_train_scaled, y_train)

    # 예측
    y_pred = model.predict(X_test_scaled)

    # 성능 저장
    results.append({
        'Model': name,

        'Accuracy':
            accuracy_score(y_test, y_pred),

        'Precision_macro':
            precision_score(
                y_test,
                y_pred,
                average='macro',
                zero_division=0
            ),

        'Recall_macro':
            recall_score(
                y_test,
                y_pred,
                average='macro',
                zero_division=0
            ),

        'F1_macro':
            f1_score(
                y_test,
                y_pred,
                average='macro',
                zero_division=0
            )
    })

# =====
# 4. DataFrame 변환
# =====

results_df = pd.DataFrame(results)

print(results_df)


# =====
# 5. 결과프레임 결과 변환
# =====

results_melted = results_df.melt(
    id_vars='Model',
    var_name='Metric',
    value_name='Score'
)

```

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from matplotlib.colors import ListedColormap

# =====
# 1. PCA 2차원 축소
# =====

pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

label_map = {
    'D': 0,
    'X': 1,
    'Z': 2
}

y_num = np.vectorize(label_map.get)(y_train)

cmap_light = ListedColormap([
    '#FFCCCC',
    '#CCFFCC',
    '#CCCCFF'
])

cmap_bold = ListedColormap([
    '#FF0000',
    '#00AA00',
    '#0000FF'
])

# =====
# 2. 사각형 판별
# =====

def plot_qda_boundary(model, title):
    model.fit(X_train_pca, y_train)

    x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
    y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 500),
        np.linspace(y_min, y_max, 500)
    )

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z_num = np.vectorize(label_map.get)(Z)
    Z_num = Z_num.reshape(xx.shape)

    plt.figure(figsize=(10, 7))

    plt.contourf(
        xx,
        yy,
        Z_num,
        alpha=0.3,
        cmap=cmap_light
    )

```



```

scatter = plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=y_train,
    cmap=cmap_bold,
    edgecolor='k',
    alpha=0.8
)

plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.title(title)

handles = scatter.legend_elements()[0]

plt.legend(
    handles,
    ['Diesel', 'Regular Gasoline', 'Premium Gasoline']
)

plt.show()

# =====
# 3. QDA 기본 모델
# =====

qda_default = QuadraticDiscriminantAnalysis()

plot_qda_boundary(
    qda_default,
    'QDA Decision Boundary - Default Prior'
)

# =====
# 4. QDA 균등 prior 모델
# =====

qda_equal_prior = QuadraticDiscriminantAnalysis(
    priors=[1/3, 1/3, 1/3]
)

plot_qda_boundary(
    qda_equal_prior,
    'QDA Decision Boundary - Equal Prior'
)

```

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.svm import SVC

from matplotlib.colors import ListedColormap

# =====
# 1. PCA 2차원 축소
# =====

pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# =====
# 2. 2차원 데이터에 SVM
# =====

svm_default = SVC(
    kernel='rbf',
    random_state=42
)

svm_default.fit(X_train_pca, y_train)

# =====
# 3. Grid 탐색
# =====

x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

# =====
# 4. Grid 예측
# =====

Z = svm_default.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

# 예측 결과 출력
label_map = {
    'D': 0,
    'X': 1,
    'Z': 2
}

Z_num = np.vectorize(label_map.get)(Z)
y_num = np.vectorize(label_map.get)(y_train)

Z_num = Z_num.reshape(xx.shape)

```

```

# =====
# 5. 색상 설정
# =====

cmap_light = ListedColormap([
    '#FFCCCC',
    '#CCFFCC',
    '#CCCCFF'
])

cmap_bold = ListedColormap([
    '#FF0000',
    '#00AA00',
    '#0000FF'
])

# =====
# 6. 시각화
# =====

plt.figure(figsize=(10, 7))

# decision boundary
plt.contourf(
    xx,
    yy,
    Z_num,
    alpha=0.3,
    cmap=cmap_light
)

# 실제 데이터
scatter = plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=y_num,
    cmap=cmap_bold,
    edgecolor='k',
    alpha=0.8
)

plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')

plt.title('Default SVM Decision Boundary')

# 레전드
handles = scatter.legend_elements()[0]

plt.legend(
    handles,
    ['Diesel', 'Regular Gasoline', 'Premium Gasoline']
)

plt.show()

```

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from matplotlib.colors import ListedColormap

# =====
# 1. PCA를 2차원 축소
# (시각화 목적)
# =====

pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# =====
# 2. SVM 학습
# =====

svm_vis = SVC(
    kernel='rbf',
    class_weight='balanced',
    random_state=42
)

svm_vis.fit(X_train_pca, y_train)

# =====
# 3. Decision Boundary Grid 생성
# =====

x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

# =====
# 4. Grid 예측
# =====

Z = svm_vis.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

# 예측 결과를 클래스 + 숫자로 변환
label_map = {
    'D': 0,
    'X': 1,
    'Z': 2
}

Z_num = np.vectorize(label_map.get)(Z)
y_num = np.vectorize(label_map.get)(y_train)

Z_num = Z_num.reshape(xx.shape)

```

```

# =====
# 5. A|B|C
# =====

plt.figure(figsize=(10, 7))

cmap_light = ListedColormap([
    '#FFCCCC',
    '#CCFFCC',
    '#CCCCFF'
])

cmap_bold = ListedColormap([
    '#FF0000',
    '#00AA00',
    '#0000FF'
])

# decision region
plt.contourf(
    xx,
    yy,
    Z_num,
    alpha=0.3,
    cmap=cmap_light
)

# training points
scatter = plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=y_num,
    cmap=cmap_bold,
    edgecolor='k',
    alpha=0.8
)

plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')

plt.title('SVM Decision Boundary (PCA 2D Projection)')

# B|C
handles = scatter.legend_elements()[0]

plt.legend(
    handles,
    ['Diesel', 'Regular Gasoline', 'Premium Gasoline']
)

plt.show()

```

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.svm import SVC

from sklearn.metrics import mean_squared_error, zero_one_loss

# =====
# y 숫자 인코딩
# =====

fuel_map = {'D': 0, 'X': 1, 'Z': 2}

y_train_num = y_train.map(fuel_map)
y_test_num = y_test.map(fuel_map)

def to_class(y_pred):
    return np.clip(np.round(y_pred), 0, 2).astype(int)

# =====
# 1. OLS / WLS: polynomial degree를 바꿔 보자
# =====

degrees = range(1, 7)

ols_train_err, ols_test_err = [], []
wls_train_err, wls_test_err = [], []

for d in degrees:
    ols = make_pipeline(
        PolynomialFeatures(degree=d, include_bias=False),
        StandardScaler(),
        LinearRegression()
    )

    ols.fit(X_train, y_train_num)

    pred_train = to_class(ols.predict(X_train))
    pred_test = to_class(ols.predict(X_test))

    ols_train_err.append(zero_one_loss(y_train_num, pred_train))
    ols_test_err.append(zero_one_loss(y_test_num, pred_test))

    sample_weight = np.ones(len(y_train_num))
    sample_weight[y_train_num == 0] = 40

    wls = make_pipeline(
        PolynomialFeatures(degree=d, include_bias=False),
        StandardScaler(),
        LinearRegression()
    )

    wls.fit(
        X_train,
        y_train_num,
        linearregression__sample_weight=sample_weight
    )

    pred_train = to_class(wls.predict(X_train))
    pred_test = to_class(wls.predict(X_test))

```

```

pred_train = to_class(wls.predict(X_train))
pred_test = to_class(wls.predict(X_test))

wls_train_err.append(zero_one_loss(y_train_num, pred_train))
wls_test_err.append(zero_one_loss(y_test_num, pred_test))

# =====
# 2. QDA: reg_param으로 범람도 조절
# reg_param 조절 = 범람
# reg_param 조절 = 단순
# =====

reg_params = np.linspace(1.0, 0.0, 10)

qda_train_err, qda_test_err = [], []

for r in reg_params:
    qda = QuadraticDiscriminantAnalysis(reg_param=r)

    qda.fit(X_train_scaled, y_train)

    qda_train_err.append(1 - qda.score(X_train_scaled, y_train))
    qda_test_err.append(1 - qda.score(X_test_scaled, y_test))

# =====
# 3. SVM: gamma로 범람도 조절
# gamma 조절 = 단순, gamma 조절 = 범람
# =====

gamma_values = np.logspace(-3, 2, 10)

svm_train_err, svm_test_err = [], []

for g in gamma_values:
    svm = SVC(
        kernel='rbf',
        gamma=g,
        class_weight='balanced',
        random_state=42
    )

    svm.fit(X_train_scaled, y_train)

    svm_train_err.append(1 - svm.score(X_train_scaled, y_train))
    svm_test_err.append(1 - svm.score(X_test_scaled, y_test))

```

```

# =====
# 그림 2.4
# =====

def plot_tradeoff(x, train_err, test_err, title, xlabel, logx=False):
    plt.figure(figsize=(8, 5))

    plt.plot(x, train_err, marker='o', label='Training Error')
    plt.plot(x, test_err, marker='o', linestyle='--', label='Test Error')

    if logx:
        plt.xscale('log')

    plt.xlabel(xlabel)
    plt.ylabel('Prediction Error')
    plt.title(title)

    plt.legend()
    plt.grid(True)
    plt.show()

# =====
# 그림 2.5
# =====

plot_tradeoff(
    list(degrees),
    ols_train_err,
    ols_test_err,
    'Bias-Variance Tradeoff: OLS',
    'Model Complexity (Polynomial Degree)'
)

plot_tradeoff(
    list(degrees),
    wls_train_err,
    wls_test_err,
    'Bias-Variance Tradeoff: WLS',
    'Model Complexity (Polynomial Degree)'
)

plot_tradeoff(
    list(range(1, len(reg_params) + 1)),
    qda_train_err,
    qda_test_err,
    'Bias-Variance Tradeoff: QDA',
    'Model Complexity (Lower Regularization → Higher Complexity)'
)

plot_tradeoff(
    gamma_values,
    svm_train_err,
    svm_test_err,
    'Bias-Variance Tradeoff: SVM',
    'Model Complexity (Gamma)',
    logx=True
)

```



```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# =====
# 1. PCA에 사용할 숫자형 변수만 선택
# (Transmission dummy 제외)
# =====

numeric_cols = [
    'Engine Size(L)',
    'Cylinders',
    'Fuel Consumption City (L/100 km)',
    'Fuel Consumption Hwy (L/100 km)',
    'CO2 Emissions(g/km)'
]

X_numeric = df[numeric_cols]

# =====
# 2. Scaling
# =====

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_numeric)

# =====
# 3. PCA 적용
# 설명분산 95% 추출
# =====

pca = PCA(n_components=0.95)

X_pca = pca.fit_transform(X_scaled)

print("원래 숫자형 변수 개수:", X_numeric.shape[1])

print("PCA 후 추출된 개수:", X_pca.shape[1])

print("추출 설명분산 비율:",
      pca.explained_variance_ratio_.sum())

# =====
# 4. PCA 결과 DataFrame
# =====

pca_columns = [
    f'PC{i+1}'
    for i in range(X_pca.shape[1])
]

df_pca = pd.DataFrame(
    X_pca,
    columns=pca_columns
)

print(df_pca.head())

```

```

# =====
# 5. PCA 설명률 설명문산 확인
# =====

explained_df = pd.DataFrame({
    'PC': pca_columns,
    'Explained Variance Ratio': pca.explained_variance_ratio_,
    'Cumulative Explained Variance': np.cumsum(pca.explained_variance_ratio_)
})

explained_df

# =====
# 6. PCA ■ VIF 확인
# =====

vif_pca = pd.DataFrame()

vif_pca['Variable'] = df_pca.columns

vif_pca['VIF'] = [
    variance_inflation_factor(df_pca.values, i)
    for i in range(df_pca.shape[1])
]

vif_pca

```

```

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd

# =====
# PCA 적용된 승차량 변수 + Transmission dummy 적용
# =====

transmission_cols = [
    col for col in df.columns
    if col.startswith('Transmission_Group_')
]

X_transmission = df[transmission_cols].reset_index(drop=True)

X_pca_final = pd.concat(
    [
        df_pca.reset_index(drop=True),
        X_transmission
    ],
    axis=1
)

y = df['Fuel Type'].reset_index(drop=True)

# =====
# Train / Test Split
# =====

from sklearn.model_selection import train_test_split

X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(
    X_pca_final,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

# =====
# SVM 모델 학습
# =====

svm_pca = SVC(
    kernel='rbf',
    class_weight='balanced',
    random_state=42
)

svm_pca.fit(X_train_pca, y_train_pca)

# =====
# 예측
# =====

y_pred_svm_pca = svm_pca.predict(X_test_pca)

```

```

# =====
# 4. 결과 출력
# =====

print("===== SVM + PCA 결과 =====")
print("Accuracy:", accuracy_score(y_test_pca, y_pred_svm_pca))
print("Precision_macro:", precision_score(y_test_pca, y_pred_svm_pca, average='macro'))
print("Recall_macro:", recall_score(y_test_pca, y_pred_svm_pca, average='macro'))
print("F1_macro:", f1_score(y_test_pca, y_pred_svm_pca, average='macro', zero_divis

print("\nClassification Report")
print(classification_report(y_test_pca, y_pred_svm_pca))

print("\nConfusion Matrix")
print(confusion_matrix(y_test_pca, y_pred_svm_pca))

import numpy as np
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from matplotlib.colors import ListedColormap

# =====
# 1. PCA 2차원 축소
# =====

pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

label_map = {
    'D': 0,
    'X': 1,
    'Z': 2
}

y_num = np.vectorize(label_map.get)(y_train)

cmap_light = ListedColormap([
    '#FFCCCC',
    '#CCFFCC',
    '#CCCCFF'
])

cmap_bold = ListedColormap([
    '#FF0000',
    '#00AA00',
    '#0000FF'
])

```

```

# =====
# 2. LDA 시각화 함수
# =====

def plot_lda_boundary(model, title):

    model.fit(X_train_pca, y_train)

    x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
    y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 500),
        np.linspace(y_min, y_max, 500)
    )

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z_num = np.vectorize(label_map.get)(Z)
    Z_num = Z_num.reshape(xx.shape)

    plt.figure(figsize=(10, 7))

    plt.contourf(
        xx,
        yy,
        Z_num,
        alpha=0.3,
        cmap=cmap_light
    )

    scatter = plt.scatter(
        X_train_pca[:, 0],
        X_train_pca[:, 1],
        c=y_num,
        cmap=cmap_bold,
        edgecolor='k',
        alpha=0.8
    )

    plt.xlabel('PCA Component 1')
    plt.ylabel('PCA Component 2')
    plt.title(title)

    handles = scatter.legend_elements()[0]

    plt.legend(
        handles,
        ['Diesel', 'Regular Gasoline', 'Premium Gasoline']
    )

    plt.show()

```

```

# =====
# 3. LDA 기본 모델
# =====

lda_default = LinearDiscriminantAnalysis()

plot_lda_boundary(
    lda_default,
    'LDA Decision Boundary - Default Prior'
)

# =====
# 4. LDA 균등 prior 모델
# =====

lda_equal_prior = LinearDiscriminantAnalysis(
    priors=[1/3, 1/3, 1/3]
)

plot_lda_boundary(
    lda_equal_prior,
    'LDA Decision Boundary - Equal Prior'
)

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

import pandas as pd

# =====
# 결과 저장 리스트
# =====

results = []

# =====
# 1. LDA 기본 prior
# =====

lda_default = LinearDiscriminantAnalysis()

lda_default.fit(X_train_scaled, y_train)

y_pred_lda_default = lda_default.predict(X_test_scaled)

results.append({
    'Model': 'LDA_Default',
    'Accuracy': accuracy_score(y_test, y_pred_lda_default),
    'Precision_macro': precision_score(
        y_test,
        y_pred_lda_default,
        average='macro'
    ),
    'Recall_macro': recall_score(
        y_test,
        y_pred_lda_default,
        average='macro'
    ),
    'F1_macro': f1_score(
        y_test,
        y_pred_lda_default,
        average='macro'
    )
})

```

```

# =====
# 2. LDA 등 prior
# =====

lda_equal = LinearDiscriminantAnalysis(
    priors=[1/3, 1/3, 1/3]
)

lda_equal.fit(X_train_scaled, y_train)

y_pred_lda_equal = lda_equal.predict(X_test_scaled)

results.append({
    'Model': 'LDA_EqualPrior',
    'Accuracy': accuracy_score(y_test, y_pred_lda_equal),
    'Precision_macro': precision_score(
        y_test,
        y_pred_lda_equal,
        average='macro'
    ),
    'Recall_macro': recall_score(
        y_test,
        y_pred_lda_equal,
        average='macro'
    ),
    'F1_macro': f1_score(
        y_test,
        y_pred_lda_equal,
        average='macro'
    )
})

# =====
# 결과 출력
# =====

results_df = pd.DataFrame(results)

display(results_df.round(6))

```

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

import pandas as pd

# =====
# Elastic Net Logistic Regression
# =====

elastic_clf = LogisticRegression(
    penalty='elasticnet',
    solver='saga',
    l1_ratio=0.5,
    C=1.0,
    multi_class='multinomial',
    max_iter=5000,
    random_state=42
)

elastic_clf.fit(X_train_scaled, y_train)

# =====
# 예측
# =====

y_pred = elastic_clf.predict(X_test_scaled)

# =====
# 성능 지표
# =====

results = pd.DataFrame([
    'Model': 'ElasticNet_Logistic',
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision_macro': precision_score(
        y_test,
        y_pred,
        average='macro'
    ),
    'Recall_macro': recall_score(
        y_test,
        y_pred,
        average='macro'
    ),
    'F1_macro': f1_score(
        y_test,
        y_pred,
        average='macro'
    )
])

display(results.round(6))

```



```
coef_df = pd.DataFrame(
    elastic_clf.coef_,
    columns=X_train.columns
)
```

```
display(coef_df)
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
```

```
# 시계열 변수 선택
```

```
plot_df = df[['Engine Size(L)', 'CO2 Emissions(g/km)', 'Fuel Type']].copy()
```

```
# 스케일링
```

```
scaler = StandardScaler()
```

```
plot_df[['Engine_Size_scaled', 'CO2_scaled']] = scaler.fit_transform(
    plot_df[['Engine Size(L)', 'CO2 Emissions(g/km)']]
)
```

```
# 산점도 시각화
```

```
plt.figure(figsize=(8,6))
```

```
sns.scatterplot(
    data=plot_df,
    x='Engine_Size_scaled',
    y='CO2_scaled',
    hue='Fuel Type',
    alpha=0.7
)
```

```
plt.title('Scatter Plot of Scaled Engine Size and CO2 Emissions by Fuel Type')
```

```
plt.xlabel('Scaled Engine Size')
```

```
plt.ylabel('Scaled CO2 Emissions')
```

```
plt.legend(title='Fuel Type')
```

```
plt.show()
```

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import zero_one_loss

# =====
# 1. LDA: shrinkage = 0.0 ~ 1.0
# shrinkage = 0.0 = 단순
# shrinkage = 1.0 = 완전
# =====

shrinkage_values = np.linspace(1.0, 0.0, 10)

lda_train_err, lda_test_err = [], []

for s in shrinkage_values:
    lda = LinearDiscriminantAnalysis(
        solver='lsqr',
        shrinkage=s
    )

    lda.fit(X_train_scaled, y_train)

    lda_train_err.append(1 - lda.score(X_train_scaled, y_train))
    lda_test_err.append(1 - lda.score(X_test_scaled, y_test))

# =====
# 2. Elastic Net Logistic Regression: C = 0.01 ~ 100
# C = 0.01 = 강하게 규제, 강하게 = 단순
# C = 100 = 약하게 규제, 약하게 = 복잡
# =====

C_values = np.logspace(-3, 2, 10)

elastic_train_err, elastic_test_err = [], []

for c in C_values:
    elastic = LogisticRegression(
        penalty='elasticnet',
        solver='saga',
        l1_ratio=0.5,
        C=c,
        class_weight='balanced',
        max_iter=5000,
        random_state=42
    )

    elastic.fit(X_train_scaled, y_train)

    elastic_train_err.append(1 - elastic.score(X_train_scaled, y_train))
    elastic_test_err.append(1 - elastic.score(X_test_scaled, y_test))

```

```

# =====
# 그림 3.10
# =====

def plot_tradeoff(x, train_err, test_err, title, xlabel, logx=False):

    plt.figure(figsize=(8, 5))

    plt.plot(x, train_err, marker='o', label='Training Error')
    plt.plot(x, test_err, marker='o', linestyle='--', label='Test Error')

    min_idx = np.argmin(test_err)
    best_x = x[min_idx]
    best_test_err = test_err[min_idx]

    plt.scatter(
        best_x,
        best_test_err,
        s=120,
        marker='^',
        label='Best Test Error'
    )

    plt.annotate(
        f'Min Test Error\n({best_x:.3f}, {best_test_err:.3f})',
        (best_x, best_test_err),
        textcoords="offset points",
        xytext=(10, -15)
    )

    if logx:
        plt.xscale('log')

    plt.xlabel(xlabel)
    plt.ylabel('Prediction Error')
    plt.title(title)

    plt.legend()
    plt.grid(True)
    plt.show()

# =====
# 그림 3.11
# =====

plot_tradeoff(
    list(range(1, len(shrinkage_values) + 1)),
    lda_train_err,
    lda_test_err,
    'Bias-Variance Tradeoff: LDA',
    'Model Complexity (Lower Shrinkage + Higher Complexity)'
)

plot_tradeoff(
    C_values,
    elastic_train_err,
    elastic_test_err,
    'Bias-Variance Tradeoff: Elastic Net Logistic Regression',
    'Model Complexity (C: Lower Regularization + Higher Complexity)',
    logx=True
)

```

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from matplotlib.colors import ListedColormap

# =====
# 1. PCA 2차원 축소
# (A) (B)
# =====

pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# =====
# 2. Elastic Net Logistic Regression
# =====

elastic_vis = LogisticRegression(
    penalty='elasticnet',
    solver='saga',
    l1_ratio=0.5,
    C=1.0,
    class_weight='balanced',
    max_iter=5000,
    random_state=42
)

elastic_vis.fit(X_train_pca, y_train)

# =====
# 3. Decision Boundary Grid
# =====

x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

```

```

# =====
# 4. Grid 예측
# =====

Z = elastic_net.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

# 문자열 클래스 + 숫자 변환
label_map = {
    'D': 0,
    'X': 1,
    'Z': 2
}

Z_num = np.vectorize(label_map.get)(Z)
y_num = np.vectorize(label_map.get)(y_train)

Z_num = Z_num.reshape(xx.shape)

# =====
# 5. 시각화
# =====

plt.figure(figsize=(10, 7))

cmap_light = ListedColormap([
    '#FFCCCC',
    '#CCFFCC',
    '#CCCCFF'
])

cmap_bold = ListedColormap([
    '#FF0000',
    '#00AA00',
    '#0000FF'
])

# decision region
plt.contourf(
    xx,
    yy,
    Z_num,
    alpha=0.3,
    cmap=cmap_light
)

# training points
scatter = plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=y_num,
    cmap=cmap_bold,
    edgecolor='k',
    alpha=0.8
)

plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')

plt.title('Elastic Net Decision Boundary (PCA 2D Projection)')

```

```
# 결과
handles = scatter.legend_elements()[0]

plt.legend(
    handles,
    ['Diesel', 'Regular Gasoline', 'Premium Gasoline']
)

plt.show()
```